Problem Statement

Title

Development of an RDF-Based Semantic Knowledge Graph for GraphRAG using Interconnected XML Documents

---

1. Background

The organization maintains a large technical knowledge repository containing more than **20,000 XML documents**. Each XML document represents one knowledge page and contains:

- XML Title
- XML Content
- XML Reference
- XML Source Path
- Document Metadata

Unlike ordinary documents, these XML files are interconnected through hyperlinks embedded within their content.

The hyperlinks appear in formats such as:

- doc:wiki:Temp_Import
- doc:Temp_Import

These hyperlinks function similarly to **Wikipedia hyperlinks**, allowing one XML document to reference another XML document.

For example,

Turbine Engineering is a team within

[[HotEnd Design>>doc:wiki:Temp_Import.HotEnd_Design.WebHome]]

Here,

- **Turbine Engineering** is one XML page.
- **HotEnd Design** is another XML page.
- The hyperlink connects these two XML documents.

However, the hyperlink itself does not explain **how** these two documents are related.

---

2. Problem

Traditional RAG systems rely mainly on:

- Text Chunking
- Embedding Generation
- Vector Database
- Similarity Search

Although vector retrieval is effective for semantic similarity, it has several limitations.

It cannot naturally understand:

- document-to-document relationships
- interconnected knowledge
- multi-hop reasoning
- dependency between technical documents

For example,

If the user asks

How is MATLAB related to Turbine Engineering?

A vector search may retrieve only the XML containing "MATLAB" or "Turbine Engineering", but it cannot automatically discover the chain of connected documents.

The XML repository already contains a rich network of hyperlinks.

Unfortunately, these hyperlinks are currently treated as plain text rather than meaningful semantic relationships.

Therefore, an approach is required that converts these interconnected XML documents into a semantic Knowledge Graph before performing retrieval.

---

3. Objective

The objective of this project is to build a **semantic RDF-based Knowledge Graph** from the XML repository and use it as the foundation of a GraphRAG system.

Instead of generating embeddings first, the system first constructs the Knowledge Graph.

The graph preserves the relationships between XML documents and later supports graph traversal, multi-hop retrieval, and semantic question answering.

---

4. Proposed Solution

The proposed solution consists of converting every XML document into a Knowledge Graph node.

The hyperlinks embedded inside the XML content are extracted and interpreted as document connections.

Instead of using the hyperlink itself as the edge, the surrounding sentence is analyzed using an LLM to determine the semantic relationship between the connected XML documents.

The extracted relationship is represented as an RDF triple.

Finally, all RDF triples are combined to create a semantic Knowledge Graph.

---

5. Knowledge Graph Design

Node

Each XML document becomes exactly **one Knowledge Node**.

One XML

↓

One Node

The node stores:

- Node ID
- XML Reference
- XML Title
- XML Content
- Source Path

Example

Node

Title:

Turbine Engineering

Reference:

Temp_Import.Turbine_Engineering.WebHome

The **Reference** serves as the unique internal identifier, while the **Title** is used for display and visualization.

Hyperlinks

Inside the XML content, hyperlinks appear in formats such as

[[HotEnd Design>>doc:wiki:Temp_Import.HotEnd_Design.WebHome]]

[[MATLAB>>doc:wiki:Temp_Import.MATLAB.WebHome]]

[[Solver>>doc:Temp_Import.Solver.WebHome]]

Only the following hyperlink types are considered:

- doc:wiki:Temp_Import
- doc:Temp_Import

These hyperlinks identify another XML document.

Edge

The hyperlink itself is **not** used as the edge.

Instead, the surrounding sentence is analyzed.

Example

Turbine Engineering is a team within

HotEnd Design.

The LLM extracts

within

The edge becomes

Turbine Engineering

|

within

▼

HotEnd Design

Another example

Engine Analysis depends on Solver.

The edge becomes

Engine Analysis

|

depends_on

▼

Solver

Thus,

- Node = XML Document
- Edge = Semantic Relationship inferred from the sentence

---

6. RDF Representation

Every relationship is stored as an RDF Triple.

Example

Subject

Temp_Import.Turbine_Engineering.WebHome

Predicate

within

Object

Temp_Import.HotEnd_Design.WebHome

The RDF Triple becomes

(

Temp_Import.Turbine_Engineering.WebHome,

within,

Temp_Import.HotEnd_Design.WebHome

)

Notice that the RDF triple stores the **XML References** internally rather than only the display titles.

This ensures that every node in the graph uniquely identifies an XML document.

---

7. Implementation Workflow

Stage 1 – Environment Setup

- Install required libraries
- Import Python packages
- Configure Azure OpenAI
- Configure Spark environment

Stage 2 – XML Selection

Read the evaluation dataset.

Identify the XML source paths.

Retrieve the matching XML documents.

Stage 3 – XML Loading

Load XML documents from the raw XML repository.

Extract:

- Document ID
- Title
- Content
- Reference
- Source Path

Stage 4 – XML Cleaning

Prepare XML content by

- removing duplicates
- replacing null values
- trimming text
- normalizing whitespace

The cleaned XML is ready for graph construction.

Stage 5 – Knowledge Node Creation

Each XML document becomes one Knowledge Node.

One XML

↓

One Knowledge Node

Each node contains

- Reference
- Title
- Content

No chunking is performed.

Each XML remains one complete knowledge unit.

Stage 6 – Hyperlink Extraction

The XML content is scanned for hyperlinks.

Supported hyperlink formats

doc:wiki:Temp_Import

doc:Temp_Import

For every hyperlink, the pipeline extracts

- Source XML Reference
- Source Title
- Target XML Reference
- Display Text
- Link Type

Example

Source

Temp_Import.Turbine_Engineering.WebHome

↓

Target

Temp_Import.HotEnd_Design.WebHome

Stage 7 – Sentence Identification

The XML content is divided into sentences.

For every extracted hyperlink, the pipeline locates the sentence containing that hyperlink.

The hyperlink markup is removed to produce a clean sentence.

Example

Original

Turbine Engineering is a team within

[[HotEnd Design>>doc:wiki:Temp_Import.HotEnd_Design.WebHome]]

Clean Sentence

Turbine Engineering is a team within HotEnd Design.

Stage 8 – Semantic Relationship Extraction

Azure OpenAI analyzes the clean sentence.

Input

Source XML

Sentence

Target XML

Example

Source

Turbine Engineering

Sentence

Turbine Engineering is a team within HotEnd Design.

Target

HotEnd Design

Output

within

The LLM extracts the semantic relationship connecting the source XML page to the linked XML page.

Stage 9 – RDF Triple Construction

The semantic relationship is converted into RDF triples.

Example

Subject

Temp_Import.Turbine_Engineering.WebHome

Predicate

within

Object

Temp_Import.HotEnd_Design.WebHome

RDF Triple

(

Temp_Import.Turbine_Engineering.WebHome,

within,

Temp_Import.HotEnd_Design.WebHome

)

Stage 10 – Knowledge Graph Construction

All RDF triples are inserted into a directed Knowledge Graph.

For every RDF triple

Subject

↓

Predicate

↓

Object

the graph performs

- Create Subject Node
- Create Object Node
- Create Directed Edge

Example

Temp_Import.Turbine_Engineering.WebHome

|

within

▼

Temp_Import.HotEnd_Design.WebHome

During visualization, these references are displayed using their XML titles.

Turbine Engineering

|

within

▼

HotEnd Design

Stage 11 – Knowledge Graph Visualization

The graph is visualized to verify

- Nodes
- Edges
- RDF triples
- Semantic relationships
- Interconnected XML documents

Final Output

The final output of the offline phase is a **semantic Knowledge Graph** built from the interconnected XML repository.

The graph contains:

- One node for every XML document.
- Directed semantic edges connecting related XML documents.
- RDF triples representing semantic relationships.
- Unique XML references as node identifiers.
- XML titles used for visualization.

Unlike a traditional hyperlink graph, the resulting graph captures the **meaning** of the connections between documents rather than only the existence of hyperlinks.

Summary

This project proposes an **RDF-based GraphRAG framework** for constructing a semantic Knowledge Graph from more than **20,000 interconnected XML documents**. Each XML document is treated as a single knowledge node, while hyperlinks (doc:wiki:Temp_Import and doc:Temp_Import) embedded within the XML content identify connections to other XML documents. The system extracts these hyperlinks, locates the corresponding sentences, and uses an LLM to infer the semantic relationship between the source and target documents. These relationships are represented as RDF triples (subject, predicate, object) using XML references as unique node identifiers. The RDF triples are then used to build a directed Knowledge Graph, where nodes represent XML documents and edges represent meaningful semantic relationships such as **within**, **uses**, **depends_on**, or **under**. This semantic graph forms the offline knowledge base for the GraphRAG system and provides the

foundation for efficient graph traversal, multi-hop reasoning, and accurate retrieval of interconnected technical knowledge before generating answers with an LLM.